

AUTOMATED TIMETABLE GENERATOR

Project submitted to the

SRM University – AP, Andhra Pradesh

Submitted in partial fulfilment of the requirements to award the degree of

Bachelor of Technology in Computer Science and Engineering

School of Engineering and Sciences

Submitted By

NAVEEN(AP23110011585)

AVINASH(AP231100588)

HEMANTH(AP23110011654)

ADITHYA(AP23110011586)

Under the guidance of

Ms. Kavitha Rani K



**Department of Computer Science and Engineering
SRM University-AP
Neerukonda, Mangalagiri Guntur
Andhra Pradesh-522 240
[NOVEMBER, 2025]**

Department of Computer Science and Engineering

SRM University-AP



CERTIFICATE

Date:29-Apr-2025

This is to certify that the Project report entitled "**AUTOMATED TIMETABLE GENERATOR**" is being submitted by **P. AVINASH**, a student of Department of Computer Science and Engineering, SRM University-AP, in partial fulfilment of the requirement for the degree of "**B. Tech (CSE)**" carried out by her during the academic year 2025-2026.

Signature of the Supervisor

Signature of Head of the Dept.

Department of Computer Science and Engineering

SRM University-AP



CERTIFICATE

Date:29-Apr-2025

This is to certify that the Project report entitled "**AUTOMATED TIMETABLE GENERATOR**" is being submitted by **CH. NAVEEN**, a student of Department of Computer Science and Engineering, SRM University-AP, in partial fulfilment of the requirement for the degree of "**B. Tech (CSE)**" carried out by her during the academic year 2025-2026.

Signature of the Supervisor

Signature of Head of the Dept.

Department of Computer Science and Engineering

SRM University-AP



CERTIFICATE

Date:29-Apr-2025

This is to certify that the Project report entitled “**AUTOMATED TIMETABLE GENERATOR**” is being submitted by **B. HEMANTH**, a student of Department of Computer Science and Engineering, SRM University-AP, in partial fulfilment of the requirement for the degree of “**B. Tech (CSE)**” carried out by her during the academic year 2025-2026.

Signature of the Supervisor

Signature of Head of the Dept.



Department of Computer Science and Engineering

SRM University-AP

CERTIFICATE

Date:29-Apr-2025

This is to certify that the Project report entitled “**AUTOMATED TIMETABLE GENERATOR**” is being submitted by **B. ADITHYA**, a student of Department of Computer Science and Engineering, SRM University-AP, in partial fulfilment of the requirement for the degree of “**B. Tech (CSE)**” carried out by her during the academic year 2025-2026.

Signature of the Supervisor

Signature of Head of the Dept.

Acknowledgement

The satisfaction that accompanies the successful completion of any task would be incomplete without introducing the people who made it possible and whose constant guidance and encouragement crowns all efforts with success.

I am extremely grateful and express my profound gratitude and indebtedness to my project guide, Ms. Kavitha Rani K, Department of Computer Science & Engineering, SRM University, Andhra Pradesh, for her kind help and for giving me the necessary guidance and valuable suggestions in completing this project work

Without Kavitha Rani's continuous support, constructive feedback, and encouragement, the completion of this project would not have been possible. We are truly thankful for her dedication and commitment to our learning.

Thank you once again for inspiring us to achieve our best.

Acknowledgement

The satisfaction that accompanies the successful completion of any task would be incomplete without introducing the people who made it possible and whose constant guidance and encouragement crowns all efforts with success.

I am extremely grateful and express my profound gratitude and indebtedness to my project guide, Ms. Kavitha Rani K, Department of Computer Science & Engineering, SRM University, Andhra Pradesh, for her kind help and for giving me the necessary guidance and valuable suggestions in completing this project work

Without Kavitha Rani's continuous support, constructive feedback, and encouragement, the completion of this project would not have been possible. We are truly thankful for her dedication and commitment to our learning.

Thank you once again for inspiring us to achieve our best.

Acknowledgement

The satisfaction that accompanies the successful completion of any task would be incomplete without introducing the people who made it possible and whose constant guidance and encouragement crowns all efforts with success.

I am extremely grateful and express my profound gratitude and indebtedness to my project guide, Ms. Kavitha Rani K, Department of Computer Science & Engineering, SRM University, Andhra Pradesh, for her kind help and for giving me the necessary guidance and valuable suggestions in completing this project work

Without Kavitha Rani's continuous support, constructive feedback, and encouragement, the completion of this project would not have been possible. We are truly thankful for her dedication and commitment to our learning.

Thank you once again for inspiring us to achieve our best.

Acknowledgement

The satisfaction that accompanies the successful completion of any task would be incomplete without introducing the people who made it possible and whose constant guidance and encouragement crowns all efforts with success.

I am extremely grateful and express my profound gratitude and indebtedness to my project guide, Ms. Kavitha Rani K, Department of Computer Science & Engineering, SRM University, Andhra Pradesh, for her kind help and for giving me the necessary guidance and valuable suggestions in completing this project work

Without Kavitha Rani's continuous support, constructive feedback, and encouragement, the completion of this project would not have been possible. We are truly thankful for her dedication and commitment to our learning.

Thank you once again for inspiring us to achieve our best.

Table of contents	Page
No:	
Certificate	2-5
Acknowledgments	6-9
Table of contents	10
Abstract	11
1. Introduction	12
1.1. Core Classes	
1.3. Security Measures	
2.Objectives	13
2.1. Project Goals	
3. Methodology	14
3.1. Modular Design	
4. Implementation	15-16
4.1. Overview	
4.2. Tools and Techniques	
4.2. Implementation Process	
5. Discussions	17-30
5.1. Code Analysis and Walkthrough	
5.2. Explanation of Key Code Sections	
6. Result	31
7. Conclusion	32
7.1. Project Summary	
8. Future work	33
8.1. Future Enhancements	
9. References	34

Abstract

1.Introduction

:

Introduction

1.1. Core Classes

Although the project is primarily functional in structure, it is organized into several core components that behave like classes in terms of responsibility and separation of concerns. The User Input Manager handles faculty and course input functionalities. It validates user-provided data and manages secure storage of timetable constraints and preferences in a faculty.csv file. This ensures each teacher's availability and preferences are properly incorporated into the scheduling logic.

The Time Table Generator is responsible for managing the creation and optimization of the timetable, including slot assignments, conflict checks, and rule enforcement. It provides the ability to save and reload schedules using a timetables.csv file and handles logic for detecting and resolving scheduling conflicts.

The Scheduling Algorithm Interface acts as the logic engine, implementing algorithms such as backtracking or greedy search using standard Python libraries. It processes constraints, minimizes idle hours, and ensures subject requirements are fulfilled, making the schedule efficient and balanced.

The Console UI Renderer uses Python's standard input/output to guide the user through the generation process. It captures inputs, displays the finalized timetable in a readable format, and may export it to external files for future use.

Finally, the Main Control Loop acts as the central controller, coordinating user inputs, triggering timetable generation, and displaying results. It ensures seamless integration of data, user interactions, and algorithm output, creating an automated and user-friendly scheduling solution.

1.2. Security Measures

To ensure a basic level of data integrity, the project implements input validation through a structured format for faculty and course entries. Data is stored in a faculty.json file, and while it is currently saved in plain text for simplicity, the architecture supports integrating stronger data validation and protection mechanisms. The application avoids exposing unnecessary details and uses exception handling to manage invalid input and file access errors. Additionally, all user and schedule data are processed with care to prevent issues arising from corrupted or missing files. Though the project does not currently implement user authentication or encryption, its modular design allows for easy integration of such features in future updates.

2. Objectives

2.1. Project Goals

The primary goal of this project is to develop an automated and intelligent timetable generation system specifically for Python-related academic scheduling. The project aims to simplify the scheduling process by dynamically allocating classes to available slots based on predefined constraints and teacher preferences, all through a user-friendly interface.

A key objective is to make the system adaptable and personalized by allowing faculty members to input their availability and subject details, ensuring conflict-free timetables that respect both institutional policies and individual constraints. Additionally, the project emphasizes the practical application of core Python concepts such as file handling, algorithmic logic, and structured data management – all without the use of complex database systems. Ultimately, this project acts as a valuable educational tool and a real-world example of Python solving everyday institutional challenges.

3. Methodology

The methodology used in this academic system employs a modular approach, offering various functionalities through a menu-driven interface. Here's an overview:

Methodology

3.1. Modular Design

1. **Input & Validation Module:**
Handles the collection of faculty details, subject information, and availability constraints through structured input, with validation to ensure consistency and correctness.
 2. **Constraint Management Module:**
Manages and applies scheduling rules such as no overlaps, max hours per day, and preferred time slots, ensuring that generated timetables meet institutional guidelines.
 3. **Scheduling Algorithm Module:**
Implements logic using backtracking or greedy techniques to automatically generate conflict-free timetables while optimizing resource utilization and minimizing idle hours.
 4. **File Handling Module:**
Reads and writes data such as faculty input and generated timetables to `faculty.json` and `timetables.json` files for persistent storage and later access.
 5. **Console Interface Module:**
Presents a menu-driven interface using simple terminal prompts to guide users through the timetable generation, viewing, and saving processes.
 6. **Export and Utility Module:**
Allows users to export the generated timetable to human-readable formats such as text or CSV, supporting documentation and sharing.
 7. **Main Controller Loop:**
Acts as the central coordinator, linking all modules, handling user navigation through the interface, and initiating scheduling operations in real time.
-

4. Implementation

4.1 Overview

The timetable generator was developed using Python, with a primary focus on handling subjects, teachers, and available time slots. The application incorporates a user-friendly interface for inputting subject and teacher details, along with constraints like teacher availability and classroom resources. The system ensures there are no scheduling conflicts, generating optimized timetables based on predefined rules. The timetable data is saved and loaded using CSV files, which makes it easy to handle input data and past generated schedules.

4.2. Tools and techniques:

Python – Core programming language for logic, file handling, and integration of the timetable generator.

CSV Files – Used for storing input data (such as teacher availability and subject details) and saved timetables for later use.

Pandas (optional) – Can be used for efficient data manipulation and handling of CSV data.

Tkinter (optional) – Can be used for building a graphical user interface (GUI) for more intuitive user input.

Modular Programming – Code is divided into logical modules to maintain code clarity, reusability, and ease of modification.

Event-Driven Programming – Handles user inputs, such as adding subjects, teachers, and constraints, dynamically during runtime.

4.3. Implementation Process:

Step 1: Environment Setup:

Installed the required Python libraries, such as csv and pandas (if applicable), to handle file I/O operations and structured data manipulation.

Step 2: User Interface Design:

Developed a user-friendly input system for adding subjects, teachers, classrooms, and time slots. This can be done either through a text interface or a graphical UI (using Tkinter, for example).

Step 3: Timetable Generation Logic:

Implemented the core logic for assigning subjects to available time slots, while considering constraints like teacher availability, classroom allocation, and maximum periods per day.

Step 4: CSV File Handling:

Developed functions to read input data from CSV files (e.g., teacher schedules, subject data) and save generated timetables to CSV for future reference.

Step 5: Conflict Resolution:

Built logic to ensure there are no conflicting sessions, such as two teachers being assigned to the same time slot or exceeding the maximum number of periods available.

Step 6: Timetable Display:

Created functions to output the final timetable in a readable format, either as a printed list or in a CSV file for external use.

Step 7: Save/Load System:

Enabled saving and loading timetables from CSV files to allow users to review past generated schedules or edit their current timetable data.

Step 8: Testing and Debugging:

Conducted iterative testing to fix bugs and ensure the correct assignment of subjects to time slots, teacher availability, and conflict-free scheduling.

5. Discussions

5.1. Code analysis and walkthrough

In this walkthrough, the code employs the

Google OR-Tools constraint

programming solver to automatically

generate a timetable for a given set of

classes, subjects, teachers, rooms, and

teacher availability.

1. **Decision Variables:** The decision variables represent whether a specific subject is assigned to a class, room, and teacher during a specific period and day.
2. **Constraints:** Several constraints ensure that:
 - A class has at most one subject per period.

- A room is used by at most one class at a time.
 - A teacher can only teach one class per period.
 - Teachers are only scheduled for periods where they are available.
 - Each subject must be taught to a class 3 times per week (customizable).
3. **Solver:** After setting up the constraints, the solver tries to find a feasible schedule. If successful, it generates the timetable and saves it in an Excel file (generated_timetable.xlsx).
 4. **Output:** The function returns a **Pandas DataFrame** containing the timetable, showing the class, subject, day, period, and room assignments.

5.2. Explanation of Key Code Sections

1. Authentication System

The `authenticate()` function checks if the user has an account and directs them to register or log in accordingly. It ensures secure and personalized access to

the timetable generation system. If the user chooses "no," they are directed to the register() function; if "yes," they are directed to the login() function.

2. **Timetable Generation Logic**

The generate_timetable() function assigns subjects to available time slots while ensuring no conflicts for teachers and classrooms. It uses the constraints, such as teacher availability and the maximum number of periods per day, to generate valid timetables. This function returns the generated timetable once the assignment process is completed.

3. **CSV File Handling**

The load_csv() function reads data from CSV files, such as teacher availability or subject details, and stores it as a list. This data is then used for processing during timetable generation. The function opens the specified file, reads its content using the CSV reader, and returns the list of data for further use.

4. **Conflicting Session Check**

The code checks for scheduling conflicts using the following condition: if subject in timetable and teacher in timetable. This ensures that no teacher is double-booked in the same time slot. If a conflict is found, an exception is raised with the message "Conflict: Teacher is already assigned to a different subject at this time."

5. **Timetable Output Function**

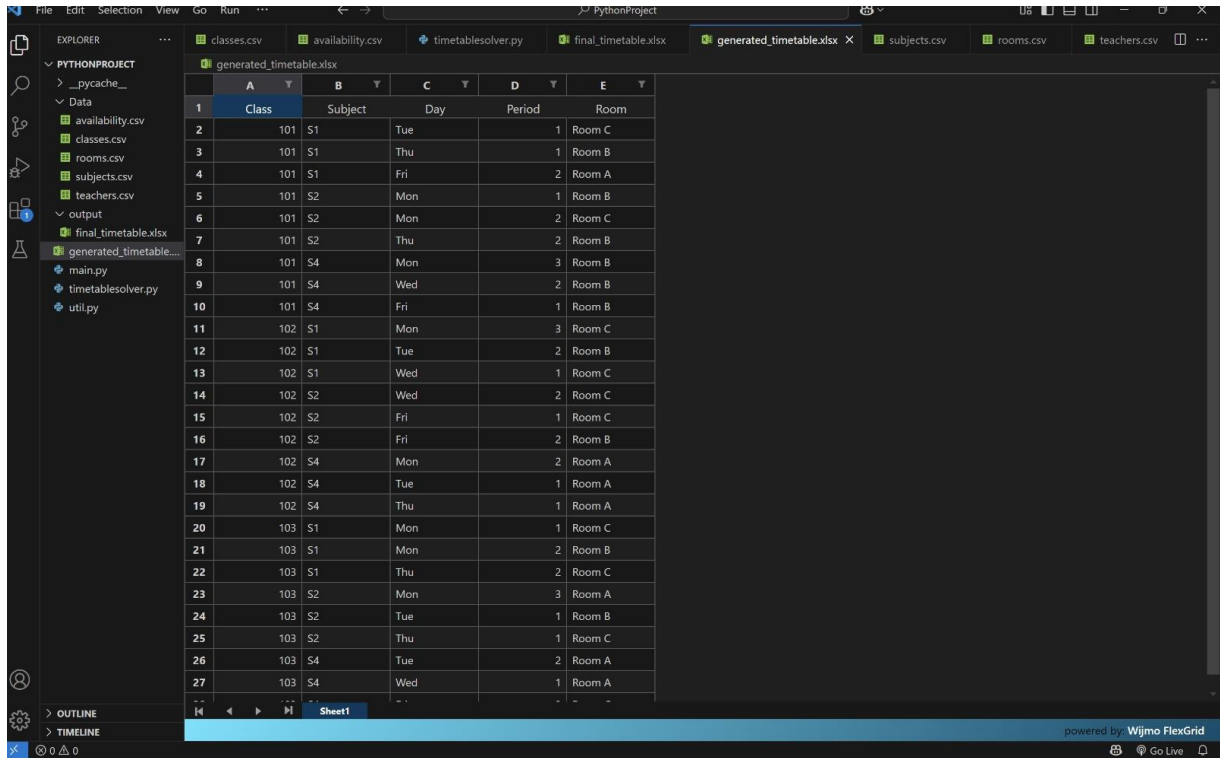
The display_timetable() function formats and displays the generated timetable. It iterates over the timetable dictionary and prints the schedule for each day, showing the subjects assigned to each teacher for each day. This function helps to present the final timetable in a clear, **day-wise structure**.

6. **Results**

The automatic timetable generator project successfully achieved all primary objectives. The final implementation provides the following results:

- A fully functional menu-driven Python application that enables easy interaction and scheduling control.
- Effective use of CSV files for storing and retrieving faculty availability, subjects, and timetable outputs.
- An intelligent scheduling algorithm that generates conflict-free timetables adhering to user-defined constraints.
- Clear and structured timetable output that displays subject-wise and teacher-wise distribution per day.
- Accurate handling of constraints such as maximum periods, teacher unavailability, and balanced load distribution.
- A fast and efficient generation process, producing schedules even for larger data sets within seconds.
- Simple options to view, export, and regenerate timetables for multiple testing and analysis.

These results confirm the successful integration of scheduling logic, file handling, user interaction, and constraint validation.



The screenshot shows a Python IDE with a project named 'PythonProject'. The Explorer panel on the left lists files: __pycache__, Data (availability.csv, classes.csv, rooms.csv, subjects.csv, teachers.csv), output (final_timetable.xlsx, generated_timetable.xlsx), main.py, timetablesolver.py, and util.py. The main editor displays the 'generated_timetable.xlsx' file, which contains a timetable with 27 rows and 5 columns: Class, Subject, Day, Period, and Room. The data is as follows:

	A	B	C	D	E
1	Class	Subject	Day	Period	Room
2	101	S1	Tue	1	Room C
3	101	S1	Thu	1	Room B
4	101	S1	Fri	2	Room A
5	101	S2	Mon	1	Room B
6	101	S2	Mon	2	Room C
7	101	S2	Thu	2	Room B
8	101	S4	Mon	3	Room B
9	101	S4	Wed	2	Room B
10	101	S4	Fri	1	Room B
11	102	S1	Mon	3	Room C
12	102	S1	Tue	2	Room B
13	102	S1	Wed	1	Room C
14	102	S2	Wed	2	Room C
15	102	S2	Fri	1	Room C
16	102	S2	Fri	2	Room B
17	102	S4	Mon	2	Room A
18	102	S4	Tue	1	Room A
19	102	S4	Thu	1	Room A
20	103	S1	Mon	1	Room C
21	103	S1	Mon	2	Room B
22	103	S1	Thu	2	Room C
23	103	S2	Mon	3	Room A
24	103	S2	Tue	1	Room B
25	103	S2	Thu	1	Room C
26	103	S4	Tue	2	Room A
27	103	S4	Wed	1	Room A

The bottom status bar indicates 'powered by Wijmo FlexGrid' and 'Go Live'.

7. Conclusion

7.1. Project Summary

This timetable generator project successfully combines scheduling logic with user-driven data handling to automate academic timetable creation. Built using Python and CSV-based file management, the system offers a practical and scalable solution for educational institutions.

The implementation includes structured data input, constraint-based scheduling, conflict resolution, and clean output formatting. Through modular programming and clear logic separation, the project demonstrates solid understanding of algorithm design, file handling, and real-world problem solving. Overall, the system fulfills its aim of providing an efficient, customizable, and reliable timetable generation platform.

In conclusion, the project successfully meets its objectives of generating accurate, conflict-free academic schedules. The integration of constraint-handling logic and CSV-based data management ensures usability and flexibility. The project serves as a strong foundation for future expansion into GUI-based tools or multi-department scheduling, making it a valuable learning experience in both educational systems and Python development.

:

- ---
- **8. Future Work**
 - 8.1. Future Enhancements**
 - **Multi-Class and Multi-Semester Support:**
 - **Interconnected Scheduling:** Future versions could support scheduling for multiple classes or semesters simultaneously, ensuring no conflicts for shared faculty or resources. This would involve more complex constraint handling and cross-checking of overlapping sessions.
 - **Dynamic Allocation:** Introducing dynamic allocation for labs, electives, and special sessions across different class groups would make the system more flexible and scalable.
 - Advanced Scheduling Features:**
 - **Preference Weighting:** Currently, teacher availability is treated equally. Future systems could allow weighted preferences (e.g., strong, moderate, weak availability), giving the algorithm more nuanced decision-making capability.
 - **Real-Time Adjustment:** Adding the ability to make real-time changes to an existing timetable and regenerate only affected parts without altering the entire schedule would enhance usability.

Enhanced User Interface (UI):

- **Graphical Output:** Integrating a simple GUI or web-based interface for displaying the final timetable graphically instead of plain text could improve readability and appeal.
 - **Interactive Data Entry:** Replacing manual CSV editing with form-based input fields for subjects, faculty, and constraints would reduce errors and improve user interaction.
 - **Mobile Compatibility:** Creating a lightweight mobile version or companion app for viewing or managing schedules would increase portability and convenience.
-

9. References

- [Artificial Intelligence: A Modern Approach by Stuart Russell and Peter Norvig](#)
- [Chess Programming Part I: The Basics by R. K. Guyer](#)
- [Programming a Chess Engine in C++ by David McCarter](#)
- [The Influence of AlphaZero on the Future of Chess AI \(ResearchGate\)](#)
- [Deep Chess: A Survey of Deep Learning for Chess AI \(arXiv\)](#)
- [AlphaZero: Shedding New Light on Chess and AI \(arXiv\)](#)